

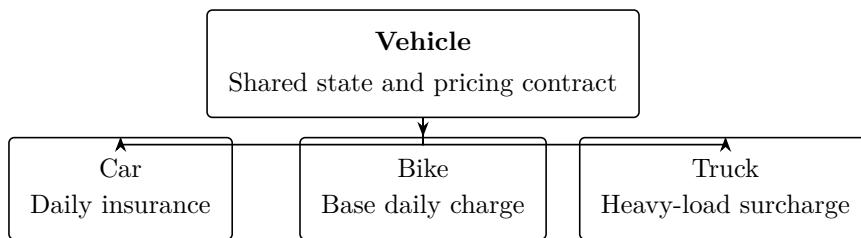
OBJECT ORIENTED PROGRAMMING

MINI PROJECT REPORT

Vehicle Rental Management System

Design, Implementation, and Verification

A dependency-free console application in Java



SUBMITTED BY

Author	Roll number
Aditya Mishra	S046
Yashas Madan	S040
Aadi Meladath	S044
Laith Shaikh	S068

Abstraction · Encapsulation · Polymorphism

September 2026

Abstract

This mini project presents a complete Vehicle Rental Management System implemented with Java Standard Edition. The system manages cars, bikes, and trucks through one console interface, while preserving the distinct pricing rules of each category. Its operations include registering vehicles, searching by identifier, listing available vehicles, renting, returning, generating immutable bills, and tracking cumulative rental revenue. A company-wide ten-percent discount applies when the rental duration exceeds seven days. All monetary values are denominated in Indian Rupees and represented using `BigDecimal` with an explicit two-decimal rounding policy.

The project's central contribution is its organization as a small, deliberate object-oriented codebase. An abstract vehicle defines common state and a pricing contract; concrete subclasses specialize charges; a rental company coordinates transactions; an immutable bill records a successful result; and a dedicated console class handles input and presentation. The entry point performs only application assembly. This organization makes state ownership, change boundaries, and error handling visible without introducing frameworks or unnecessary architectural layers.

The report explains both the implementation and the reasoning behind it. It derives the pricing equations, specifies invariants, traces successful and unsuccessful transactions, examines encapsulation and substitution limits, and analyzes operational complexity. The project README records 113 dependency-free checks passing against both the modular development version and the standalone source release. Four scripted console scenarios produced identical output, and the release was compiled independently in an empty directory. These observations support correctness for the documented scenarios; they are not a formal proof or a large-scale performance evaluation.

The report is intended to support implementation review and oral presentation. Its diagrams, worked examples, method-level explanations, demonstration sequence, and viva questions connect Java concepts directly to the delivered code.

Keywords: Java; object-oriented programming; runtime polymorphism; encapsulation; vehicle rental; immutable billing; decimal arithmetic; single-file distribution.

How to Use This Report

Read Chapters 1–2 first to establish the problem and the responsibility boundaries. Chapters 3–6 then follow the data from construction through pricing, rental, billing, and terminal output. Chapters 7–9 explain why the design is structured this way and what evidence supports it. The final chapters discuss costs, limitations, and a practical demonstration.

Presentation question	Where to look
-----------------------	---------------

What does the project do?	Problem, scope, and use cases in Chapter 1.
Where does each rule live?	Architecture in Chapter 2; classes in Chapters 3 and 4.
How does a rental work?	Transaction sequence and failure boundary in Chapter 4.
Why are the numbers correct?	Equations and rounding in Chapter 5.
Where is OOP demonstrated?	Dynamic dispatch and design principles in Chapter 7.
How was it checked?	Reproducible build commands and verification in Chapters 8 and 9.
How should we demonstrate it?	Guided walkthrough in Chapter 11; viva answers in Appendix A.

Reading conventions. Class and method names appear in monospace. A figure’s arrow means a call or dependency unless its legend explicitly says otherwise. UML-style hollow-triangle arrows point toward a superclass. Values marked INR are sample application amounts. “Revenue” means the sum of accepted rental charges in this program, not a record of actual payment settlement.

Evidence boundary. The report is grounded in the project’s README and modular Java sources, including the updated INR console. It is a technical mini-project report, not a claim of new scientific research, a production financial system, or an implemented database application. Proposed extensions are explicitly separated from delivered behavior.

Contents

Abstract	i
How to Use This Report	ii
1 Problem, Objectives, and Scope	1
1.1 The problem being solved	1
1.2 Functional requirements	1
1.3 Software engineering objectives	2
1.4 Use-case boundary	2
1.5 Scope and assumptions	2
2 Architecture and Responsibility Boundaries	3
2.1 Four conceptual responsibilities	3
2.2 Ownership is the organizing principle	3
2.3 The class relationship model	3
2.4 Why the design stays small	3
3 Domain Model: State, Contracts, and Value Objects	6
3.1 Vehicle: the common abstraction	6
3.1.1 State and construction	6
3.1.2 Identity normalization	6
3.1.3 The method contract	6
3.2 Availability as a state machine	7
3.3 Car: pricing with insurance	8
3.4 Bike: pricing without a surcharge	8
3.5 Truck: pricing with a heavy-load surcharge	8
3.6 RentalBill: an immutable result	9
3.6.1 Data captured	9
3.6.2 Validation and consistency	9
4 RentalCompany and Transaction Correctness	10
4.1 Fleet and revenue state	10

4.2	The six public service operations	10
4.2.1	addVehicle	10
4.2.2	findVehicleById	10
4.2.3	listAvailableVehicles	10
4.2.4	rentVehicle	11
4.2.5	returnVehicle	11
4.2.6	getTotalRevenue	12
4.3	Successful rental sequence	12
4.4	The failure boundary	12
5	Pricing, Discounts, and Exact Money	14
5.1	The mathematical model	14
5.2	Why BigDecimal is appropriate	14
5.3	The rounding sequence	15
5.4	Boundary and category examples	15
5.5	Revenue as an invariant	16
6	Console Interface and Input Recovery	17
6.1	A focused presentation class	17
6.2	Menu control flow	17
6.3	Line-based parsing	17
6.4	Rate input and business validation	18
6.5	Formatting methods and their boundaries	18
6.6	Error handling strategy	19
7	Object-Oriented Principles and Design Rationale	20
7.1	Abstraction and dynamic dispatch	20
7.2	Encapsulation beyond private fields	20
7.3	Why inheritance is justified here	21
7.4	SOLID, applied to this code	21
7.4.1	Single Responsibility Principle	21
7.4.2	Open/Closed Principle	21
7.4.3	Liskov Substitution Principle	21
7.4.4	Interface Segregation Principle	22
7.4.5	Dependency Inversion Principle	22

7.5	Other principles	22
7.6	Decision record	22
8	Implementation, Build, and Single-File Release	24
8.1	Development layout	24
8.2	Bootstrap and sample data	24
8.3	Build and run commands	25
8.4	How the standalone release works	25
8.5	Keeping the two representations consistent	26
8.6	Java portability versus report typesetting	26
9	Verification Strategy and Evidence	28
9.1	What the evidence establishes	28
9.2	Test class organization	28
9.3	Representative verification matrix	28
9.4	Why failure tests are necessary	30
9.5	Release independence and parity	30
9.6	Remaining verification limits	30
10	Complexity and Future Scalability	31
10.1	Cost model	31
10.2	Why no Map is implemented now	31
10.3	What changes at production scale	32
11	Demonstration and Presentation Walkthrough	33
11.1	A concise explanation of the project	33
11.2	Suggested ten-minute presentation structure	33
11.3	Live demonstration sequence	33
11.4	The five-day bike example	34
11.5	Explaining a rejected duplicate	34
11.6	Explaining extension without implementing it	35
12	Limitations, Future Work, and Conclusion	36
12.1	Deliberate limitations	36
12.2	Future work with concrete design consequences	36
12.3	Conclusion	37

A Viva and Presentation Questions	38
B Glossary and Source Navigation	41
B.1 Where to find an implementation detail	42
C Requirement-to-Design Traceability	43
References	44

List of Figures

1.1	The operator interacts with one local application; no external service participates.	2
2.1	Responsibility layers and the direction of business dependencies. Bootstrap also constructs the company and concrete vehicles.	3
2.2	Inheritance, association, and result creation. An ordinary association is used for fleet references; exclusive cross-company ownership is not enforced.	4
3.1	Valid transitions are solid arrows. Dashed loops represent rejected operations, not successful self-transitions.	7
3.2	A bill is a value snapshot, not a view onto a mutable vehicle.	9
4.1	Collection encapsulation is shallow: sorting the derived container is isolated, but vehicle objects are shared.	11
4.2	Sequence of a successful rental. Calls are solid; returned values are dashed. The bill is prepared before either persistent in-memory state change.	12
4.3	The service prepares all fallible business calculations before the commit section. . .	13
5.1	One pricing pipeline feeds both the bill and revenue, preventing competing totals. .	14
6.1	Format recovery happens in the UI; business validation remains below it.	18
7.1	Runtime polymorphism removes category-specific pricing conditionals from the service.	20
8.1	Single-file distribution preserves multiple-class design.	26
9.1	Verification proceeds from behavior to interaction and distribution independence. .	29
11.1	The intended extension boundary, shown as a proposed future category rather than delivered functionality.	35

List of Tables

- 1.1 Functional requirements and observable acceptance conditions. 1
- 2.1 Each important rule has a deliberate owner. 4
- 3.1 Vehicle API and visibility. 7
- 5.1 Worked rounding example, in INR. 15
- 5.2 Representative prices in INR. The strict boundary is greater than seven days. . . . 16
- 6.1 The menu switch routes interaction; it does not implement pricing. 17
- 7.1 Major design decisions, alternatives, and consequences. 23
- 8.1 Initial fleet and illustrative INR daily rates. 25
- 9.1 Required cases and recorded outcomes. All monetary values are INR. 29
- 10.1 Operation costs with respect to fleet size. 31
- 11.1 A suggested presentation order; the team may divide speaking roles as preferred. . . 33
- 12.1 Possible extensions and the new responsibilities they would introduce. 36
- B.1 Key terms in the context of this project. 41
- C.1 Traceability from requirement to implementation and verification evidence. 43

Problem, Objectives, and Scope

1.1 The problem being solved

A rental company has to answer three related questions: which vehicles exist, which may be rented now, and what a valid rental should cost. These questions become more interesting when categories have different charges. A car's insurance and a truck's heavy-load fee must be included consistently without making the rental workflow depend on every concrete type.

The system must also reject invalid operations. A nonexistent vehicle cannot be rented; a rented vehicle cannot be rented again; an available vehicle cannot be returned; and a duration of zero or a negative number of days is meaningless. Correctness therefore involves both successful calculations and the preservation of state when an operation fails.

The project solves this problem through a single-user terminal application. The operator enters a menu choice and the necessary values. The service performs the operation and returns a result or an explicit exception. The interface presents that outcome in a readable form and continues accepting input.

1.2 Functional requirements

ID	Capability	Acceptance condition
F1	Add vehicles	A valid, available car, bike, or truck with a unique normalized ID joins the fleet.
F2	List availability	Only available vehicles are listed, ordered by base daily rate and then ID.
F3	Search by ID	A known vehicle is returned whether available or rented; a missing match is represented by an empty optional.
F4	Rent	Positive duration, known identity, and availability are required before billing and mutation.
F5	Apply pricing	Runtime subtype determines the subtotal; the company applies the common discount.
F6	Issue a bill	A successful rental returns immutable vehicle, duration, subtotal, discount, and total data.
F7	Return	A rented vehicle becomes available; no revenue is added or removed.
F8	Track revenue	The final discounted total is added exactly once per successful rental.
F9	Recover from input errors	Invalid formats and expected business failures produce useful messages without terminating the menu.

Table 1.1. Functional requirements and observable acceptance conditions.

1.3 Software engineering objectives

The program is small enough that a single large method could appear to solve it. That approach would hide the ownership of rules, make changes risky, and weaken its educational value. Instead, the project aims for clear responsibilities, validated construction, immutable identity, controlled transitions, exact monetary calculations, and a narrow service API.

Extensibility is specific rather than unlimited: a new vehicle subtype should provide its own pricing without changing the common rental algorithm. Maintainability is equally concrete: changing a discount should affect the company, while changing terminal alignment should affect only the menu. These are the change scenarios used to evaluate the design.

1.4 Use-case boundary

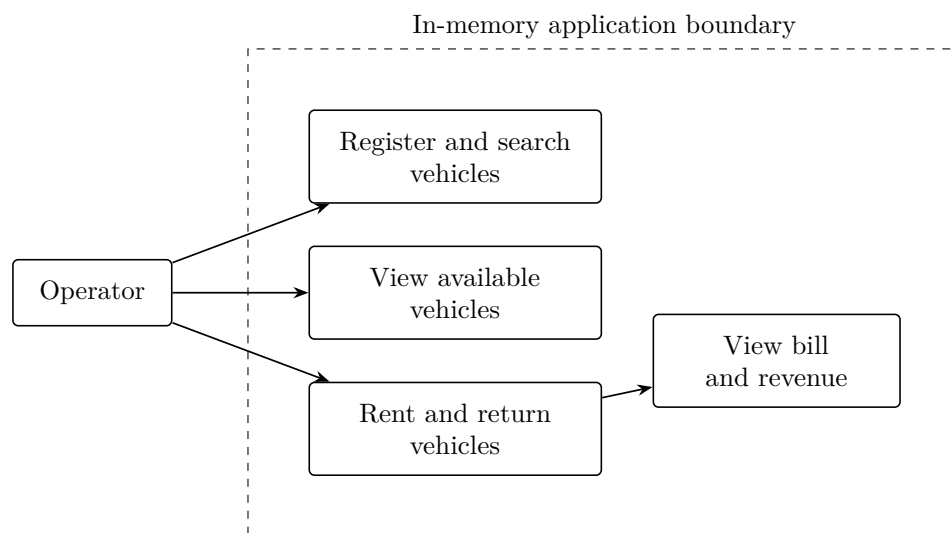


Figure 1.1. The operator interacts with one local application; no external service participates.

1.5 Scope and assumptions

The fleet is held in memory for one process. There is one operator, no concurrent access, no customer-account system, no booking calendar, and no persistent transaction history. Rental duration is a positive integer rather than the difference between stored dates. Billing occurs when a rental is accepted, not on return.

The sample rates are illustrative INR values. No exchange-rate conversion, tax calculation, payment transfer, or legal invoice generation is implemented. A new run constructs a fresh fleet with all vehicles available and revenue equal to INR 0.00. These boundaries keep the project focused on object-oriented design.

Presentation takeaway. Describe the application as a validated in-memory rental workflow with category-specific pricing. Its strength is the explicit ownership of rules, not the number of features.

Architecture and Responsibility Boundaries

2.1 Four conceptual responsibilities

The architecture separates domain modeling, application coordination, presentation, and bootstrap. These are conceptual layers in ordinary Java classes, not framework modules. Their direction of dependency determines which classes must change together.

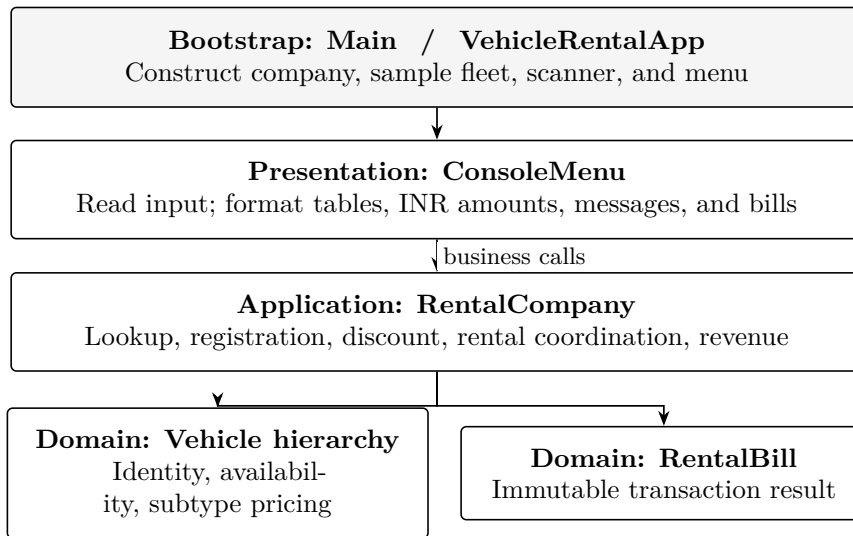


Figure 2.1. Responsibility layers and the direction of business dependencies. Bootstrap also constructs the company and concrete vehicles.

The domain knows nothing about terminal input. The service knows nothing about column widths or menu numbers. The menu can use domain queries for display, but it delegates business operations to the company. The bootstrap class owns startup composition without implementing rental logic.

2.2 Ownership is the organizing principle

2.3 The class relationship model

2.4 Why the design stays small

There is one service rather than a service interface plus a matching implementation, one list rather than a list and a secondary index, and one console class rather than a factory for each prompt. These choices avoid indirection that has no present user requirement. At the same time, separating a bill from the company and pricing from the console prevents unrelated rules from being combined.

Owner	State or rule owned	Reason for change
Vehicle	ID, model, daily rate, availability; construction and transition rules	Common rentable-vehicle behavior changes.
Car / Bike / Truck	Category-specific subtotal formula	A category's fee or calculation changes.
RentalCompany	Fleet, uniqueness, discount, accumulated revenue	A fleet-level use case or company policy changes.
RentalBill	Immutable description and monetary result	The information required in a completed rental changes.
ConsoleMenu	Input/output references and presentation rules	Interaction, formatting, or input recovery changes.
Main / VehicleRentalApp	Startup assembly	Sample data or entry-point composition changes.

Table 2.1. Each important rule has a deliberate owner.

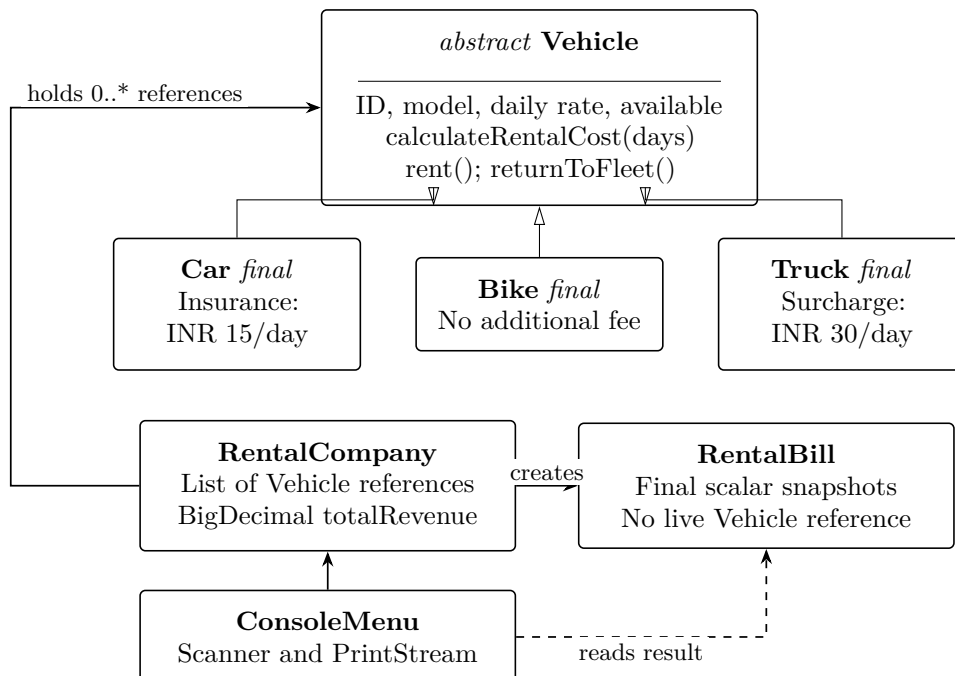


Figure 2.2. Inheritance, association, and result creation. An ordinary association is used for fleet references; exclusive cross-company ownership is not enforced.

A helpful review question is: “If this rule changes, which file should I open?” The answer should be predictable. For insurance, open `Car.java`; for discount, `RentalCompany.java`; for INR alignment, `ConsoleMenu.java`. The architecture is effective because these answers are clear.

Presentation takeaway. Walk down Figure 2.1: the menu handles conversation, the company coordinates the use case, the vehicle computes its subtotal, and the bill carries the result.

Domain Model: State, Contracts, and Value Objects

3.1 Vehicle: the common abstraction

3.1.1 State and construction

`Vehicle` has four fields: `final String id`, `final String model`, `final BigDecimal dailyRate`, and mutable `boolean available`. Availability starts as `true`. A final field prevents reassignment of its reference; in this case `String` and `BigDecimal` also provide immutable values, so identity and price cannot be edited indirectly through their returned references.

Construction validates the object before it becomes usable. ID and model must not be null or blank. The daily rate must not be null or negative. Negative values are rejected before rounding, so `-0.001` is not accidentally accepted as zero. A valid rate is normalized to two decimals with `RoundingMode.HALF_UP`.

```
protected Vehicle(String id, String model, BigDecimal dailyRate) {
    this.id = normalizeId(id);
    this.model = requireText(model, "Model");
    if (dailyRate == null || dailyRate.signum() < 0) {
        throw new IllegalArgumentException("Daily rate must be non-negative.");
    }
    this.dailyRate = dailyRate.setScale(2, RoundingMode.HALF_UP);
}
```

Listing 3.1. The shared vehicle constructor, from the development source.

3.1.2 Identity normalization

`normalizeId` strips leading and trailing Java whitespace and uppercases using `Locale.ROOT`. The same operation is called during construction and lookup. Thus " car-001 " and "CAR-001" identify the same stored object. Using a locale-independent conversion prevents the system's default language settings from changing the matching policy.

Normalization does not remove internal spaces or perform Unicode canonical normalization. Uniqueness means uniqueness of the normalized stored string within one company. It does not mean that every visually similar Unicode label is treated as identical. This distinction states the actual implemented contract precisely.

3.1.3 The method contract

Table 3.1. Vehicle API and visibility.

Method	Meaning
<code>getId()</code> , <code>getModel()</code> , <code>getDailyRate()</code>	Public final queries returning the immutable descriptive values.
<code>isAvailable()</code>	Public final query exposing current state without permitting assignment.
<code>rent()</code>	Package-private final transition. Rejects a vehicle that is already rented, otherwise sets availability false.
<code>returnToFleet()</code>	Package-private final transition. Rejects an already available vehicle, otherwise sets availability true.
<code>getType()</code>	Public abstract display-name contract. It does not control pricing.
<code>calculateRentalCost(int)</code>	Public abstract subtotal contract: positive days, nonnegative two-decimal result, no rental mutation.
<code>costWithDailyFee(int, BigDecimal)</code>	Protected final helper: validate days, add a supplied daily fee, multiply by days. Supplied subclasses pass trusted nonnegative, cent-exact constants.
<code>validateDays(int)</code>	Package-private static rule shared by the service, pricing helper, and bill construction.
<code>normalizeId(String)</code>	Package-private static identity normalization and validation.
<code>requireText(String, String)</code>	Package-private static nonblank validation and whitespace stripping, with a meaningful error label.

The protected pricing helper does not independently validate every possible fee a future subclass might pass. Consequently, extension authors must preserve the documented fee and subtotal contract. The bill validates the returned money before a successful transaction is committed, but that defensive check is not permission for a subtype to return arbitrary invalid results.

3.2 Availability as a state machine

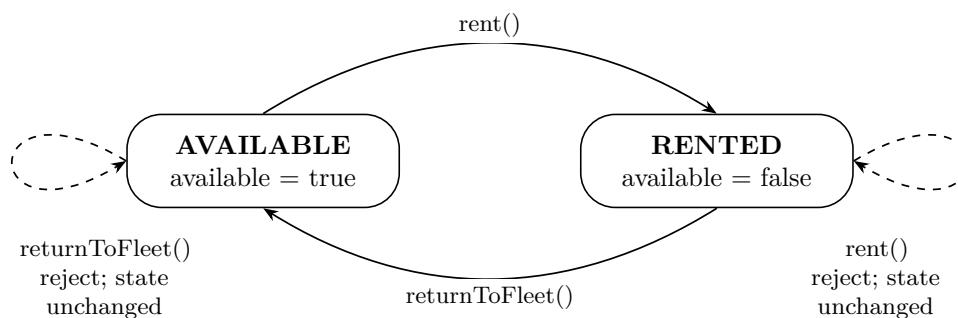


Figure 3.1. Valid transitions are solid arrows. Dashed loops represent rejected operations, not successful self-transitions.

Arbitrary setters would let callers bypass this model. For example, `setAvailable(true)` says nothing about whether a return is legitimate. By contrast, `returnToFleet()` expresses intent and enforces the precondition at the owner of the state. Making transitions final also prevents a subtype from redefining the same operation to mean something incompatible.

3.3 Car: pricing with insurance

`Car` is final and introduces one named constant, `DAILY_INSURANCE_FEE`, constructed from the exact decimal string "15.00". Its constructor delegates shared state to `super`. Its display method returns "`Car`". Its pricing override passes the daily insurance fee to the common helper.

```
@Override
public BigDecimal calculateRentalCost(int days) {
    return costWithDailyFee(days, DAILY_INSURANCE_FEE);
}
```

Listing 3.2. Car's entire pricing specialization.

For a base rate of INR 50.00, a three-day rental costs $(50 + 15) \times 3 = 195$. Insurance is charged once per day, not once per transaction. The override returns the subtotal only; it does not decide whether a long-rental discount applies.

3.4 Bike: pricing without a surcharge

`Bike` is final and has no additional fields. Its `getType()` returns "`Bike`", and its pricing override calls `costWithDailyFee(days, BigDecimal.ZERO)`. A common helper avoids copying duration validation and multiplication while still allowing the runtime subtype to own the pricing decision.

For BIKE-001 at INR 20.00 per day and five days, the subtotal is INR 100.00. Five days do not qualify for the company discount, so the final bill is also INR 100.00. A zero-rate bike is valid and produces a zero-charge rental; it still follows normal availability transitions.

3.5 Truck: pricing with a heavy-load surcharge

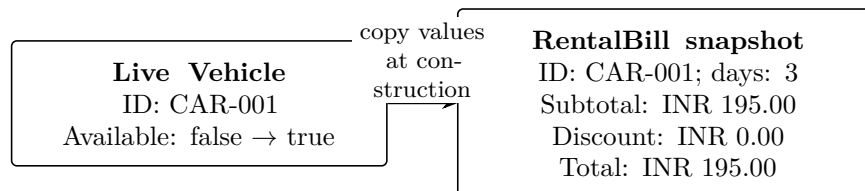
`Truck` is final and owns `DAILY_HEAVY_LOAD_SURCHARGE`, an exact INR 30.00 per day. The constructor delegates shared validation, the display method returns "`Truck`", and the pricing method calls the helper with the surcharge. At a base rate of INR 120.00, a three-day truck subtotal is $(120 + 30) \times 3 = 450$.

The service does not need to know whether the additional daily charge represents insurance or heavy-load handling. It receives a subtotal from the abstraction and applies the shared policy. That distinction is the practical value of polymorphism in this project.

3.6 RentalBill: an immutable result

3.6.1 Data captured

The bill stores final values for vehicle ID, model, type, duration, subtotal, discount, and total. It copies the descriptive values from the vehicle during construction and retains no vehicle reference. This means a later return cannot change a previous bill's data. Since the stored strings and decimals are immutable, the getters do not expose mutable components.



Returning the live vehicle changes availability.
The already-created bill remains unchanged.

Figure 3.2. A bill is a value snapshot, not a view onto a mutable vehicle.

3.6.2 Validation and consistency

The package-private constructor requires a vehicle, validates positive days, checks that its type label is nonblank, and validates subtotal and discount. Both amounts must be nonnegative and exactly representable at two decimals. The `UNNECESSARY` rounding mode accepts values such as `1.000` by removing redundant zeros, but rejects a value such as `1.001` that would require a changed amount.

Discount must not exceed subtotal. Total is calculated as subtotal minus discount inside the bill rather than being accepted as a third independent monetary input. This prevents a caller from supplying mutually inconsistent values such as subtotal 100, discount 10, and total 95.

The getters correspond to the information printed by the console. There are no setters, input operations, fleet searches, or revenue updates. The constructor is package-private to make normal creation a cooperating-service responsibility. Code in the same package could still construct a bill; the application convention is that a successful transaction result is obtained through `rentVehicle`.

Presentation takeaway. Vehicle encapsulates legal state and shared data; each subtype owns its subtotal; `RentalBill` freezes the accepted result. These are three different responsibilities even though they participate in the same rental.

RentalCompany and Transaction Correctness

4.1 Fleet and revenue state

The company declares `private final List<Vehicle> fleet = new ArrayList<>();` and initializes `totalRevenue` to `new BigDecimal("0.00")`. The final list reference cannot be reassigned, but the collection remains mutable so that vehicles can be registered. Only the company changes its membership.

Revenue is a `BigDecimal` value whose reference is reassigned after a successful transaction. The monetary object itself is immutable. No stored customer, date, payment, or bill-history collection is present.

4.2 The six public service operations

4.2.1 addVehicle

The method rejects null, searches for the proposed normalized ID, rejects duplicates, verifies the object is currently available, and appends it to the fleet. The constructor has already ensured that its identity and rate are valid. Rejecting a rented addition prevents the company from receiving a vehicle whose unavailable state was created outside its ordinary registration flow.

This method guarantees uniqueness within the list. It does not enforce a global registry across different `RentalCompany` objects. The delivered bootstrap creates one company and distinct objects, so that larger ownership problem is outside the current scope.

4.2.2 findVehicleById

The method normalizes the requested ID once and loops through the fleet. For the first matching ID, it returns `Optional.of(vehicle)`. If there is no match, it returns `Optional.empty()`. Null and blank IDs are invalid arguments, not ordinary absent results.

An optional distinguishes two situations: a valid search with no result and an invalid search request. Callers such as the search screen can display a no-match message. Operations requiring an existing object use the private `requireVehicle` helper, which converts absence into `IllegalArgumentException` with the requested identifier.

4.2.3 listAvailableVehicles

The company creates a new list, traverses its fleet, and adds references whose `isAvailable()` is true. The console then sorts this derived list by `getDailyRate()` and `getId()`. Returning a new list protects collection membership and master ordering without forcing the service to know a display preference.

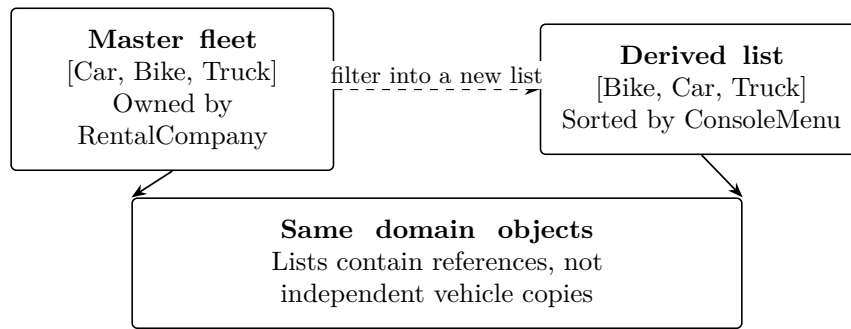


Figure 4.1. Collection encapsulation is shallow: sorting the derived container is isolated, but vehicle objects are shared.

Clearing the derived list does not clear the company fleet. However, a returned vehicle reference still denotes the same mutable vehicle. The public API exposes only availability queries; package-private transition access remains available to trusted cooperating code. This is a deliberate boundary, not a claim that a mutable object becomes deeply immutable merely because its container was copied.

4.2.4 rentVehicle

The method first validates positive days. It then finds the requested vehicle and rejects an unavailable one before calculating any charge. Runtime polymorphism produces the subtotal, company policy produces the discount, and a candidate bill validates and derives the final total. Only after all these steps does the method change availability and assign updated revenue.

```

public RentalBill rentVehicle(String id, int days) {
    Vehicle.validateDays(days);
    Vehicle vehicle = requireVehicle(id);
    if (!vehicle.isAvailable()) {
        throw new IllegalStateException(
            "Vehicle " + vehicle.getId() + " is already rented.");
    }
    BigDecimal subtotal = vehicle.calculateRentalCost(days);
    BigDecimal discount = days > LONG_RENTAL_THRESHOLD_DAYS
        ? subtotal.multiply(LONG_RENTAL_DISCOUNT_RATE)
        : new BigDecimal("0.00");
    RentalBill bill = new RentalBill(vehicle, days, subtotal, discount);
    BigDecimal updatedRevenue = totalRevenue.add(bill.getTotal());
    vehicle.rent();
    totalRevenue = updatedRevenue;
    return bill;
}
  
```

Listing 4.1. The actual rental transaction, showing preparation before mutation.

4.2.5 returnVehicle

Returning requires an existing vehicle and delegates to `returnToFleet()`. The vehicle owns the guard against a duplicate return. The method neither recalculates the earlier rental nor modifies

revenue. The previous charge was accepted at rental time, and this application's return operation is an inventory transition only.

4.2.6 getTotalRevenue

The query returns the accumulated `BigDecimal`. Since the returned value is immutable, a caller cannot edit the company's current revenue through it. Computing a new decimal using the returned value also does not assign that result back into the company.

4.3 Successful rental sequence

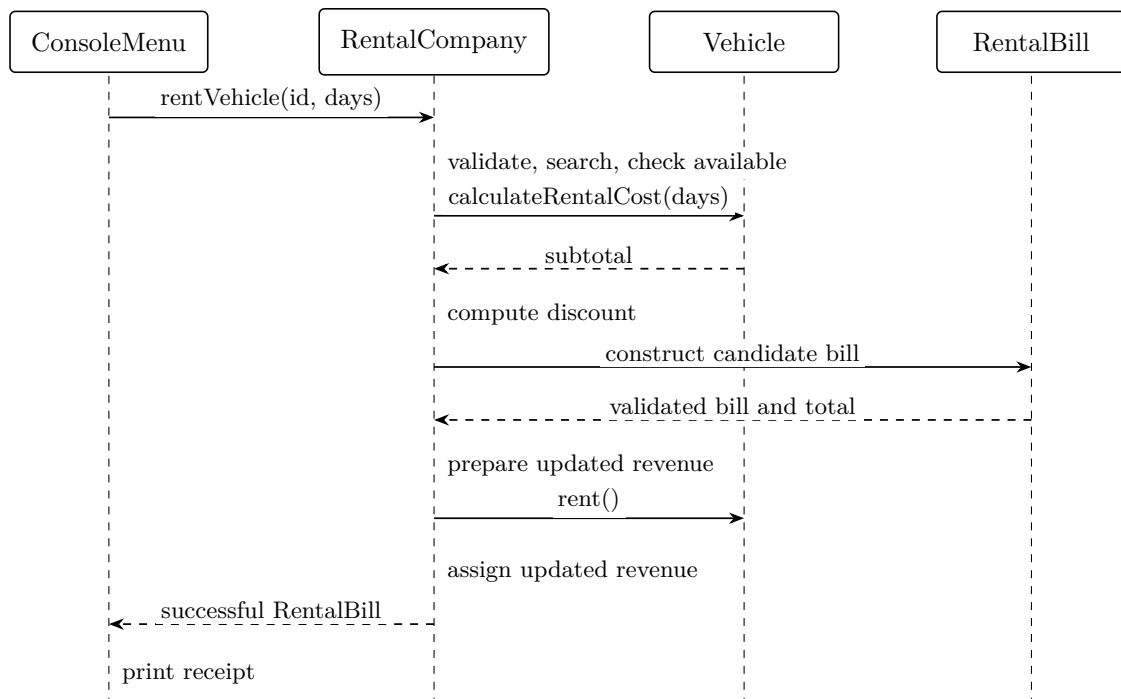


Figure 4.2. Sequence of a successful rental. Calls are solid; returned values are dashed. The bill is prepared before either persistent in-memory state change.

4.4 The failure boundary

The ordering in Listing 4.1 matters. If bill validation were performed after making the vehicle unavailable, an invalid subtotal could leave the vehicle rented without a returned bill. If revenue were changed before checking availability, a duplicate rental could create an incorrect charge. Preparation before mutation avoids these ordinary partial-operation outcomes.

This is a single-threaded application-level guarantee for the supplied contracts. It is not a database transaction, crash-recovery mechanism, or concurrency lock. A malicious subtype that changes state while calculating a price would violate the contract. A future concurrent application would require atomic coordination around availability and revenue, rather than assuming sequential statements alone are sufficient.

The service checks availability early, and `Vehicle.rent()` checks it again. These checks serve different positions in the flow: the early check avoids calculation for a known invalid rental, while

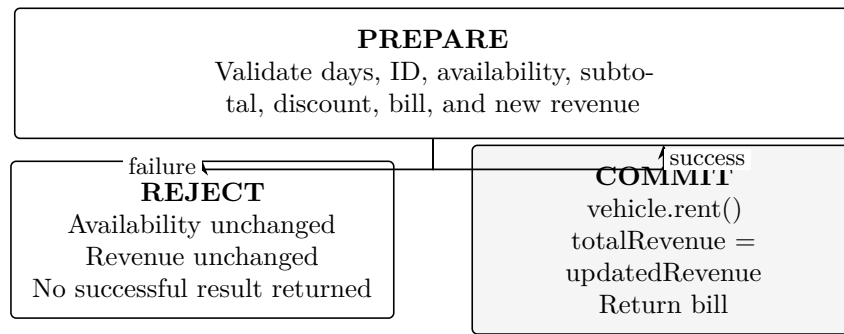


Figure 4.3. The service prepares all fallible business calculations before the commit section.

the final transition method continues to protect the state wherever it is called. The duplicated observation does not move ownership of the transition away from the vehicle.

Presentation takeaway. Explain a rental as prepare, commit, return. All validation and monetary work is prepared first; only then does the vehicle become rented and revenue increase.

Pricing, Discounts, and Exact Money

5.1 The mathematical model

Let r be a vehicle's normalized base daily rate, d the positive integer number of rental days, $i = 15.00$ the car's daily insurance, and $s = 30.00$ the truck's daily surcharge. Each subtype computes a subtotal S :

$$S_{\text{bike}}(r, d) = rd, \quad (5.1)$$

$$S_{\text{car}}(r, d) = rd + id = (r + i)d, \quad (5.2)$$

$$S_{\text{truck}}(r, d) = rd + sd = (r + s)d. \quad (5.3)$$

Every value is in INR. The subtotal includes all applicable daily fees. The company then calculates a discount D and final total T :

$$D(S, d) = \begin{cases} \text{round}_{2, \text{HALF_UP}}(0.10S), & d > 7, \\ 0.00, & 1 \leq d \leq 7, \end{cases} \quad (5.4)$$

$$T = S - D. \quad (5.5)$$

The discount includes the fee component because it is applied to the entire subtotal. A car and a truck do not calculate a second discount inside their overrides. The named company constants are `LONG_RENTAL_THRESHOLD_DAYS` and `LONG_RENTAL_DISCOUNT_RATE`.

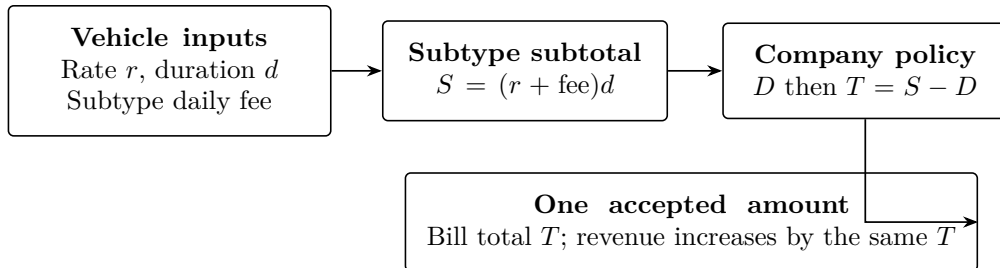


Figure 5.1. One pricing pipeline feeds both the bill and revenue, preventing competing totals.

5.2 Why BigDecimal is appropriate

Binary floating-point numbers cannot represent many finite decimal fractions exactly. Repeated financial arithmetic using those approximations can require careful compensation and formatting to avoid unexpected results. `BigDecimal` represents decimal values with an arbitrary-precision unscaled integer and a scale, and supplies explicit rounding operations [2]. It therefore fits the project's two-decimal monetary model.

Choosing `BigDecimal` does not automatically make every calculation financially meaningful. The program still needs rules about when to round, which amount is discounted, and when revenue

is recognized. Those rules are stated here and enforced by the classes that own them. No monetary operation converts a value to `float` or `double`.

Constants and sample rates are constructed from strings such as `new BigDecimal("0.10")`, so the intended decimal is exact at the input boundary. Integer durations are converted with `BigDecimal.valueOf(days)` before multiplication. No precision-limiting `MathContext` is supplied, so the additions and multiplications used here remain exact until an explicit scale adjustment.

5.3 The rounding sequence

The constructor normalizes rates to two decimals once. For nonnegative amounts, `HALF_UP` rounds a midpoint away from zero, so 1.005 becomes 1.01. Daily fees are already cent-exact, and multiplying a two-decimal value by an integer does not introduce additional fractional places requiring rounding. The service rounds the calculated percentage discount to two decimals. Finally, it subtracts that rounded discount from the subtotal.

The bill uses `UNNECESSARY` as a check, not as a competing rounding policy. It verifies that the monetary values presented to it are already exactly representable at two decimals. A future subtype returning a fractional-cent subtotal fails before mutation rather than being silently altered by the result object.

Stage	Value	Explanation
Entered bike rate	1.005	Exact decimal input before normalization.
Stored daily rate	1.01	Constructor applies two-decimal <code>HALF_UP</code> .
Eight-day subtotal	8.08	1.01×8 .
Unrounded discount	0.808	8.08×0.10 .
Rounded discount	0.81	Service applies two-decimal <code>HALF_UP</code> .
Final total	7.27	$8.08 - 0.81$.

Table 5.1. Worked rounding example, in INR.

The order of operations is part of the business definition. “Round the discount, then subtract” should not be casually replaced by “multiply subtotal by 0.9, then round” because rounding identities do not hold for every amount and midpoint. Centralizing the calculation makes that choice inspectable.

5.4 Boundary and category examples

The seven-day example is essential because an incorrect condition `days >= 7` would give a discount one day too early. The eight-day example establishes that the threshold actually activates. Testing both sides distinguishes the specified condition from nearby but incorrect implementations.

Vehicle	Days	Base/day	Subtotal	Discount	Total
Bike	3	20.00	60.00	0.00	60.00
Bike	5	20.00	100.00	0.00	100.00
Car	3	50.00	195.00	0.00	195.00
Truck	3	120.00	450.00	0.00	450.00
Bike	7	20.00	140.00	0.00	140.00
Bike	8	20.00	160.00	16.00	144.00
Car	8	50.00	520.00	52.00	468.00

Table 5.2. Representative prices in INR. The strict boundary is greater than seven days.

5.5 Revenue as an invariant

Let $R_0 = 0.00$, and let T_k denote the final charge of the k th successful rental. After m successful rentals,

$$R_m = \sum_{k=1}^m T_k. \quad (5.6)$$

An unsuccessful operation contributes no term. A return contributes no term. Renting a returned vehicle again creates a new successful rental and therefore adds a new final total. For a car rented for three days, a bike rented for eight days, and a truck rented for three days,

$$R = 195.00 + 144.00 + 450.00 = \text{INR } 789.00. \quad (5.7)$$

The amount added is obtained from `bill.getTotal()`, which makes the receipt and accounting refer to one accepted result. The system does not store real payments, so this revenue model is a simplified internal charge total.

Presentation takeaway. Explain the price in two steps: the vehicle supplies the subtotal, and the company applies the promotion. Then identify exactly where rounding happens and why revenue uses the bill's final total.

Console Interface and Input Recovery

6.1 A focused presentation class

`ConsoleMenu` is final and owns references to `RentalCompany`, `Scanner`, and `PrintStream`. Its constructor rejects null collaborators through `Objects.requireNonNull`. Its public `run()` method starts the interaction, while private methods separate menu rendering, each action, parsing, and formatting.

The startup title is printed once. The recurring menu is compact and uses two columns. Available vehicles are shown in an aligned table with text on the left and monetary columns on the right. The table states INR once above the amounts, while bills, search details, and revenue use the INR prefix explicitly. The design avoids color, cursor control, screen clearing, and Unicode box characters, making it suitable for ordinary terminals and copied output.

6.2 Menu control flow

Choice	Handler	Delegated operation
1	<code>showAvailableVehicles()</code>	Filter through company, sort derived list, print table.
2	<code>handleRental()</code>	Read ID and days, call <code>rentVehicle</code> , print returned bill.
3	<code>handleReturn()</code>	Read ID, call <code>returnVehicle</code> , confirm success.
4	<code>showRevenue()</code>	Read <code>totalRevenue</code> and format INR amount.
5	<code>handleAddVehicle()</code>	Read category and details, construct subtype, call <code>addVehicle</code> .
6	<code>handleSearch()</code>	Read ID, inspect Optional, print details or no-match message.
0	Exit branch	End the loop and print goodbye.

Table 6.1. The menu switch routes interaction; it does not implement pricing.

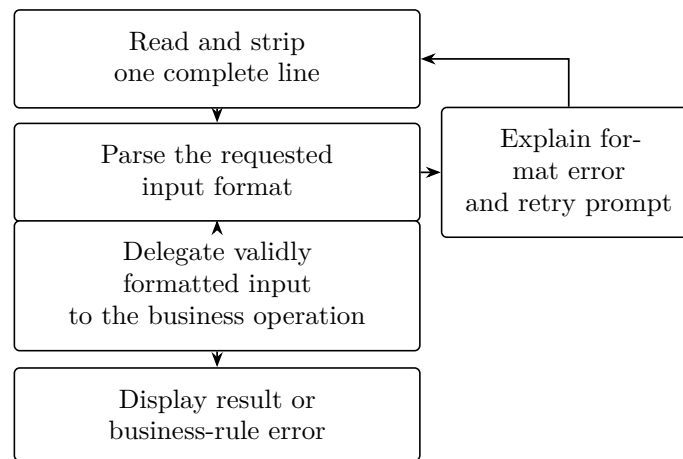
The switch in `handleAddVehicle()` chooses a constructor for Car, Bike, or Truck. This is different from selecting a pricing formula using a type code: once the object exists, the service always invokes the polymorphic method. A future category would require an added construction choice in the UI, while the rental workflow could remain unchanged.

6.3 Line-based parsing

One primary scanner is created over `System.in`. Every prompt reads a complete line, strips outer whitespace, and only then attempts conversion. `readInteger()` uses `Integer.parseInt`;

malformed text and integer overflow produce `NumberFormatException`, which the method catches before repeating the same prompt.

This avoids the familiar problem of mixing token-oriented `nextInt()` with line-oriented `nextLine()`, where a leftover line terminator can be mistaken for the next answer. The scanner lifecycle is simple: bootstrap creates and closes it; the menu uses it.



End of input at any prompt exits cleanly.
No business call is made for an unfinished rental input sequence.

Figure 6.1. Format recovery happens in the UI; business validation remains below it.

6.4 Rate input and business validation

`readDailyRate()` accepts a plain decimal string such as `50.00`, with an optional sign. Its format expression rejects letters, commas, currency symbols, and scientific notation. A negative value may be syntactically valid but remains invalid business input; the vehicle constructor rejects it without adding an object to the fleet.

This separation is intentional. Format validation asks whether input can be understood as the requested representation. Business validation asks whether the interpreted value is permitted. The same business constraints must apply to callers that never use the console, including tests and future interfaces.

6.5 Formatting methods and their boundaries

`printVehicleTable()` builds rows of display strings, including a header. It measures the maximum length in each column, constructs a format string, prints a rule under the headings, and prints each vehicle. It does not truncate IDs or models, so long input can widen the output beyond a narrow terminal. That is a transparent limitation of this small console design, not a reason to change the fleet model.

`printVehicle()` presents one search result as labeled fields, including availability. The available-vehicle table does not repeat “Available” in every row because its filtering rule already guarantees that state. Both views obtain the fee-inclusive daily price by calling `calculateRentalCost(1)`. Neither repeats the insurance or surcharge arithmetic.

`printSection()` supplies a consistent heading and rule. `printDetail()` aligns descriptive labels, and `printCharge()` right-aligns the formatted monetary result. `formatAmount()` uses `String.format` with `Locale.ROOT` and the format `"%,.2f"` directly with a decimal object. `formatMoney()` adds "INR ". Grouping uses the formatter's locale-neutral three-digit grouping, not the Indian lakh/crore grouping convention. The denomination is nevertheless INR throughout.

6.6 Error handling strategy

Expected business failures are caught as `IllegalArgumentException` or `IllegalStateException` around a menu operation. The menu prints the explanatory message and remains available. `NoSuchElementException` at an input prompt is handled as end of input, allowing a clean exit. The code does not use a broad `catch (Exception)` to hide unrelated defects.

A programming fault such as a malformed subtotal from an invalid extension is not treated as ordinary user input. The delivered subtypes satisfy the contract, while tests deliberately introduce one broken subtype to check failure before mutation. This distinction keeps user recovery friendly without suppressing failures a developer needs to see.

Presentation takeaway. The console translates human input into service calls and translates results into readable output. It does not own business rules merely because it is the first place a user encounters them.

Object-Oriented Principles and Design Rationale

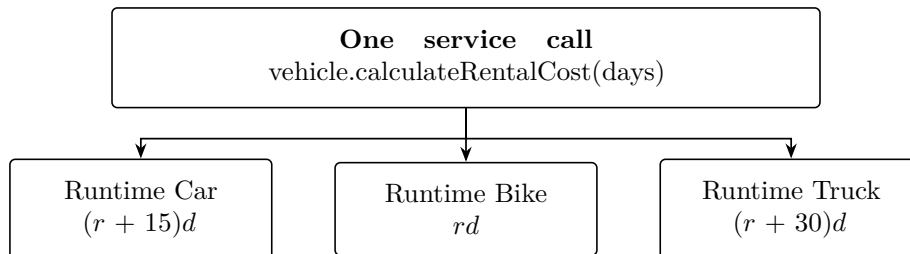
7.1 Abstraction and dynamic dispatch

An abstract class expresses a common model with some behavior left deliberately incomplete. Here, all rentable vehicles have identity, a model, a rate, and availability, but their subtotal calculation varies. Declaring `calculateRentalCost` abstract forces concrete categories to supply that behavior.

```
Vehicle vehicle = new Truck("T-1", "Atlas Cargo", new BigDecimal("120.00"));
BigDecimal subtotal = vehicle.calculateRentalCost(3);
// subtotal is 450.00: the Truck override is selected at runtime.
```

Listing 7.1. One base reference, runtime-specific behavior.

The compile-time type `Vehicle` determines which operations are legal at the call site. The runtime object `Truck` determines which override executes. Java’s runtime method lookup for an instance call supports this behavior [5]. The company therefore makes one call without knowing every concrete pricing formula.



Exactly one override runs for a particular object.
The figure shows alternative runtime
types, not three calculations per rental.

Figure 7.1. Runtime polymorphism removes category-specific pricing conditionals from the service.

7.2 Encapsulation beyond private fields

The essential question is not simply whether fields are private, but whether collaborators can violate the intended state model through the available operations. Immutable IDs prevent an accepted registration from changing identity after the uniqueness check. Guarded transitions prevent invalid availability changes. Derived lists protect fleet membership and order. Immutable bill values preserve a transaction result.

The program deliberately has no `setId`, `setModel`, `setDailyRate`, or `setAvailable`. Returning a reference from a getter is safe for strings and decimals because their values are immutable. Returning a new list is safe for collection membership but does not clone the contained vehicles. These distinctions are important when explaining what encapsulation does and does not guarantee.

7.3 Why inheritance is justified here

Car, Bike, and Truck are genuinely kinds of rentable Vehicle. They share validated identity and the same available/rented state machine. The hierarchy is shallow, and variation is concentrated in a small pricing contract. This makes inheritance a direct representation of the domain rather than a device for reusing unrelated code.

Concrete categories are final because they were not designed as additional extension points. A new category should extend Vehicle and satisfy its contract. If future requirements make pricing vary independently of category, configurable policy objects could become appropriate; they are not required for the current three fixed rules.

7.4 SOLID, applied to this code

7.4.1 Single Responsibility Principle

Each class has a primary reason to change. A formatting requirement modifies ConsoleMenu, an insurance rule modifies Car, a long-rental policy modifies RentalCompany, and a new result field modifies RentalBill and its consumers. Moving these responsibilities into Main or merely relocating them all into RentalCompany would make unrelated concerns change together.

SRP does not mean one method per class. ConsoleMenu legitimately has several formatting and input methods because they contribute to one presentation responsibility. Similarly, RentalCompany has multiple use-case operations because they coordinate one fleet and its accounting.

7.4.2 Open/Closed Principle

The rental workflow is open to a new valid subtype through the Vehicle contract without changing its pricing call, discount rule, or revenue assignment. Tests demonstrate a custom subtype using the existing workflow. This is the relevant extension boundary.

The whole application is not closed to every edit. To expose a new category to an operator, the bootstrap or add-vehicle constructor choice must be updated. Claiming that “adding a class requires no other edits anywhere” would overstate the implementation. The more precise claim is that the central rental algorithm does not need another pricing branch.

7.4.3 Liskov Substitution Principle

A supplied subtype can be used wherever a Vehicle is expected because it accepts the same positive duration domain, rejects non-positive durations, returns a nonnegative two-decimal subtotal, and does not rent or return itself during calculation. Final shared state methods preserve the meaning of transitions across categories.

A subtype that rejects a normal positive duration, changes availability while quoting, or returns an invalid subtotal would violate substitution. The protected helper makes the common implementation straightforward, but Java's type system alone does not prove every semantic contract. Extension authors and tests must preserve these requirements.

7.4.4 Interface Segregation Principle

The project uses a compact `Vehicle` abstraction and a small application API. It does not introduce `Rentable`, `Billable`, or `VehicleManagerInterface` solely to claim that interfaces were used. Collaborators receive the operations they actually need through the existing class boundaries. Avoiding unnecessary interfaces is consistent with keeping client contracts focused.

7.4.5 Dependency Inversion Principle

The company relies on the `Vehicle` abstraction for category behavior and on the `List` abstraction for collection operations. Business code does not depend on the console. This protects business decisions from presentation details.

The UI still depends directly on a concrete `RentalCompany`. The report therefore does not claim that every dependency has been inverted behind an interface. With one service implementation and no competing adapters requiring a service contract, another interface would add indirection without solving a present variation problem.

7.5 Other principles

DRY. Identity normalization, duration validation, subtype fees, company discount, and revenue assignment each have an owner. Daily display totals call the pricing API instead of duplicating its formulas. Development and release bodies are mechanically assembled from the same implementation.

KISS and YAGNI. A list, a straightforward lookup, standard exceptions, and ordinary objects are enough for the current scope. There are no factories per category, repository wrappers without persistence, service locators, plugin registries, or authentication layers. Simplicity here means fewer unnecessary concepts, not a refusal to separate real responsibilities.

Tell, Don't Ask. The company tells a vehicle to perform a transition rather than retrieving state and assigning a boolean setter. Queries remain useful for display and early validation. This application follows the behavior-with-data emphasis described by Fowler without treating every query as undesirable [7].

High cohesion and low coupling. Category charges stay together, company accounting stays together, and input recovery stays together. Collaborators use method contracts rather than editing each other's fields. Composition connects the menu to a company, the company to vehicles, and the workflow to a result object.

7.6 Decision record

Table 7.1. Major design decisions, alternatives, and consequences.

Decision	Alternative considered	Why the current choice fits
Abstract Vehicle	One class with type strings and pricing switches	Type-specific behavior is expressed by actual subtype overrides.
Final concrete types	Deep inheritance chains	Existing categories have a stable meaning; extension belongs at the base contract.
Immutable identity	Public setters after registration	Uniqueness remains meaningful after the initial check.
Guarded availability	Arbitrary boolean setter	Transition preconditions cannot be skipped through the public API.
BigDecimal	Floating-point money	Decimal inputs and explicit scale rules are represented directly.
List with ArrayList	Additional map index from the start	Current scale favors clarity over maintaining parallel structures.
Linear lookup	Premature indexing abstraction	Search, duplicate checks, and use cases remain easy to inspect.
Service discount	Same percentage copied into every subclass	One company policy has one executable owner.
Immutable bill	Mutable bill referring to live vehicle state	A completed rental result cannot drift during later operations.
Prepare then commit	Mutation before bill validation	Expected failures preserve revenue and availability.
Separate menu	Menus and Scanner inside business classes	Presentation changes and tests do not require changes to pricing.
Single-file release	Copy several files or collapse into one large class	Copying is easy while logical classes remain separate.
No framework	Container, build tool, and third-party dependencies	Standard compilation is sufficient and transparent for academic review.

Presentation takeaway. Tie every principle to a concrete change or failure scenario. Do not merely expand the SOLID acronyms; show which class protects which rule and acknowledge the limits of each claim.

Implementation, Build, and Single-File Release

8.1 Development layout

The Java application uses standard-library imports only. It has eight production source files and one development-only test source. No package declaration or module descriptor is required. The README is the project guide, and the release directory contains the self-contained distribution source.

```
vehicle-rental/
  README.md
  src/
    Vehicle.java
    Car.java
    Bike.java
    Truck.java
    RentalBill.java
    RentalCompany.java
    ConsoleMenu.java
    Main.java
    RentalCompanyTest.java
  releases/
    VehicleRentalApp.java
```

Listing 8.1. The Java project layout; this report is a separate documentation artifact.

The minimum source/API baseline is Java 11 because the implementation uses methods including `String.isBlank()`, `String.strip()`, and `String.repeat()`. The README records execution on Eclipse Temurin OpenJDK 25.0.3 and compilation targeting Java 11. It does not claim a separate execution on a Java 11 runtime.

8.2 Bootstrap and sample data

Main has a private constructor and a static main method. It constructs a `RentalCompany`, populates a useful fleet through a private helper, creates one `Scanner` in a try-with-resources block, and runs `ConsoleMenu`. The scanner is closed by that block when the application exits.

```
public static void main(String[] args) {
    RentalCompany company = new RentalCompany();
    populateSampleFleet(company);
    try (Scanner input = new Scanner(System.in)) {
        new ConsoleMenu(company, input, System.out).run();
    }
}
```

}

Listing 8.2. Bootstrap contains assembly rather than business logic.

ID	Fictional model	Base/day	With fees/day
CAR-001	Aster Compact	50.00	65.00
CAR-002	Meridian Sedan	75.00	90.00
BIKE-001	Swift City 125	20.00	20.00
BIKE-002	Trail Explorer 250	30.00	30.00
TRUCK-001	Atlas Cargo	120.00	150.00
TRUCK-002	Titan Hauler	160.00	190.00

Table 8.1. Initial fleet and illustrative INR daily rates.

The sample fleet is large enough to make heterogeneous behavior visible. Its size and identifiers are not hard-coded into the company’s loops or pricing. Additional valid vehicles enter through the same registration method.

8.3 Build and run commands

From the project directory, compile the development version and start it:

```
cd src
javac *.java
java Main
```

To run the dependency-free checks from the same directory:

```
java RentalCompanyTest
```

The checks throw `AssertionError` explicitly, so the Java `-ea` switch is unnecessary. Compilation produces class files locally; they are not part of the delivered source archive. The application requires no external JAR files or manifest.

8.4 How the standalone release works

A Java source file may contain multiple top-level classes. Under the conventional file-based compilation arrangement used by `javac`, the public class is named to match its file, while additional top-level classes can be package-private [4]. Thus `VehicleRentalApp.java` contains one public entry point plus the same seven collaborating domain, service, and UI classes.

The assembly transformation collects unique imports, renames `Main` and its constructor to `VehicleRentalApp`, removes the top-level public modifier from the other application classes, and concatenates their bodies. The test source is not included. There is no separate redesign of the release behavior.

To run it from the release directory or any directory containing only the copied source:

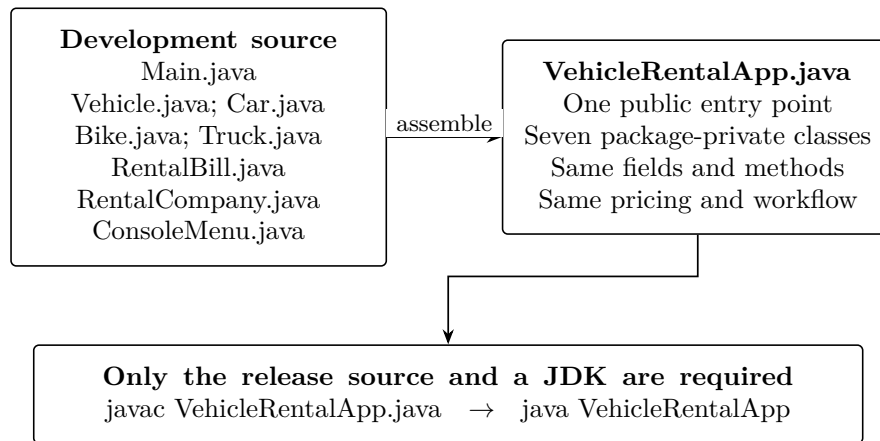


Figure 8.1. Single-file distribution preserves multiple-class design.

```

javac VehicleRentalApp.java
java VehicleRentalApp
  
```

Several class files are created during compilation because several classes still exist. “Single file” refers to the source that must be copied, not to the number of runtime class files.

8.5 Keeping the two representations consistent

Development changes must be made in `src` first. The release is then assembled again and checked independently. The two directories should not be compiled together because they intentionally declare classes with the same names in the unnamed package. Their public visibility differs at the top-level distribution boundary, but their internal application logic remains the same.

The README supplies an optional PowerShell assembly snippet. It combines source text rather than adding a runtime dependency. Its assumptions are narrow: known class names, conventional declarations, and explicit imports. If those assumptions change, the assembly process must be reviewed rather than treated as a general Java parser.

8.6 Java portability versus report typesetting

The Java system requires only a JDK. This document is a separate LaTeX artifact and uses a standard TeX distribution with PDFLaTeX and TikZ to produce a PDF. Its body font is Latin Modern Roman, a Computer Modern-family serif supplied through standard TeX font packages. This PDFLaTeX-compatible choice preserves the requested Computer Modern appearance without requiring the OpenType CMU Serif font or a different typesetting engine. These typesetting dependencies are not Java application dependencies.

All report diagrams are vector instructions embedded in this single source, and the bibliography is embedded as well. Compile this report with PDFLaTeX twice so contents, figure lists, and references resolve:

```

pdflatex -interaction=nonstopmode -halt-on-error paper.tex
pdflatex -interaction=nonstopmode -halt-on-error paper.tex
  
```

No external image files, bibliography database, custom system-font installation, or shell escape are needed. The source uses ordinary packages available in standard MiKTeX and TeX Live distributions.

Presentation takeaway. The modular form makes the project easy to maintain; the release makes it easy to copy. Logical architecture is preserved even when physical source files are combined.

Verification Strategy and Evidence

9.1 What the evidence establishes

The README records 113 checks passing for both forms of the program, four matching console-output scenarios, and two independent clean release compilations. The reported environment was Temurin OpenJDK 25.0.3 on 15 September 2026, with development compilation against the Java 11 baseline. These checks exercise expected outcomes and failure preservation, not just startup.

The results are project verification evidence, not measured code-coverage percentages, performance benchmarks, or a mathematical proof. No coverage tool or load test was run or claimed. The test class is standard Java and remains absent from the standalone release.

9.2 Test class organization

`RentalCompanyTest` is final, has a private constructor, and exposes a static entry point. Its `checks` counter records completed assertions. Six focused scenario methods organize the checks:

1. `testPricing`: category totals, invalid durations, discount boundaries, rounding, zero-rate rentals, and a large positive duration.
2. `testValidation`: construction rules, normalized IDs, duplicate registration, empty searches, and invalid additions.
3. `testTransactions`: successful rental, rejected duplicate rental, returns, unchanged revenue on failures, and immutable bill values.
4. `testCollectionOwnership`: derived-list sorting and clearing do not change the master collection.
5. `testPolymorphismAndFailure`: a new subtype works through the service and a broken subtotal is rejected before mutation.
6. `testConsole`: malformed input, business failures, bill output, additions, sorting, empty fleets, explicit exit, and end-of-input recovery.

The helpers construct exact decimals, compare values, verify exception types, and throw `AssertionError` if an expectation fails. Money comparisons use `BigDecimal.equals`, which checks both numerical value and scale; the suite therefore checks the intended two-decimal representation, not just numerical equality [2].

9.3 Representative verification matrix

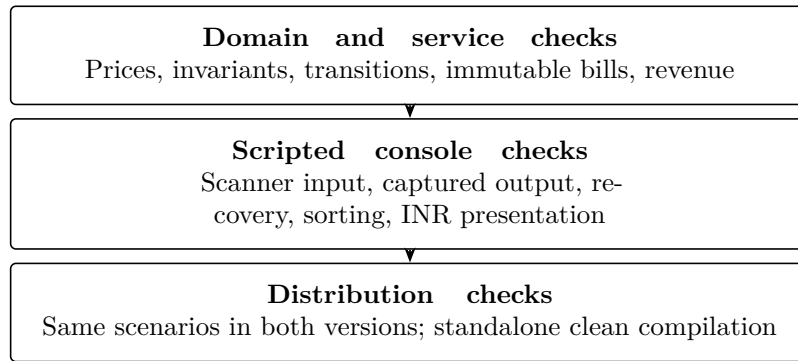


Figure 9.1. Verification proceeds from behavior to interaction and distribution independence.

Table 9.1. Required cases and recorded outcomes. All monetary values are INR.

Scenario	Expected and observed behavior	Result
Bike, 3 days at 20/day	Subtotal 60.00; no discount; total 60.00.	Pass
Car, 3 days at 50/day	Includes 45.00 insurance; total 195.00.	Pass
Truck, 3 days at 120/day	Includes 90.00 surcharge; total 450.00.	Pass
Bike, exactly 7 days	Total 140.00; discount 0.00.	Pass
Bike, 8 days	Subtotal 160.00; discount 16.00; total 144.00.	Pass
Unknown rental ID	Rejected without changing availability or revenue.	Pass
Zero or negative days	Rejected by service and each supplied subtype.	Pass
Second rental before return	Rejected; vehicle remains rented; revenue unchanged.	Pass
Return rented vehicle	Available again and listed; revenue unchanged.	Pass
Second return	Rejected without changing state or revenue.	Pass
Mixed-case, padded ID	Same object found; duplicate normalized ID rejected.	Pass
Sort or clear listing result	Master list membership and order preserved.	Pass
Fractional rate 1.005	Stored 1.01; eight-day total 7.27 after rounding.	Pass
Zero-rate bike	Valid zero-charge rental; state still transitions.	Pass
Earlier bill after return	Original vehicle description and amount unchanged.	Pass
New subtype	Custom price and company discount work through Vehicle reference.	Pass
Broken fractional subtotal	Bill rejects before availability or revenue mutation.	Pass
Malformed numeric input	Relevant prompt recovers without crashing.	Pass
End of input mid-operation	Clean exit; unfinished rental does not commit.	Pass
Combined demonstration	Car 195.00 + Bike 144.00 + Truck 450.00 = 789.00.	Pass

9.4 Why failure tests are necessary

A test that only checks the first rental's price does not show what happens when the same vehicle is rented again. A complete transaction check captures availability and revenue, attempts the invalid operation, verifies the exception, and then verifies that both values remain unchanged. The suite performs these checks for the important invalid operations.

Likewise, successful sorting does not prove collection encapsulation. The test deliberately sorts a returned list and clears it, then reads the company again to verify that its membership and insertion order remain intact. The custom subtype test checks the intended extension boundary directly rather than merely observing that three built-in types happen to work.

9.5 Release independence and parity

The release file was copied into an otherwise empty directory, compiled, and run with no modular source present. Generated class files were deleted, and compilation and execution were repeated. This rules out an accidental dependency on stale compiled classes for those runs.

After independence was established, the optional test source was compiled against the release's classes and passed the same 113 checks. Four full console scripts also produced matching output between Main and VehicleRentalApp. A mechanical comparison checked that the release bodies matched the modular sources after the declared import, entry-point, and visibility transformations.

9.6 Remaining verification limits

The tests do not establish behavior under concurrent calls, hostile extension code, process termination during mutation, unbounded input size, or external storage failures. Those situations are outside the implemented scope. Future requirements in those areas would need new designs and corresponding tests rather than a larger count of nearly identical happy-path assertions.

Presentation takeaway. Present the evidence in layers: exact price checks, state-preservation checks, console recovery checks, and a clean standalone build. State the observed result without overstating what the suite proves.

Complexity and Future Scalability

10.1 Cost model

Let n denote fleet size and a the number of available vehicles. The following bounds describe growth with fleet size, assuming typical bounded-length identifiers and monetary values. String comparison, decimal arithmetic, and output size have their own operand-length costs.

Operation	Time	Extra space	Dominant work
Add vehicle	$O(n)$	$O(1)$ amortized	Search for duplicate, then append.
Search by ID	$O(n)$	$O(1)$	Linear traversal.
Filter available	$O(n)$	$O(a)$	Traverse and build a new list.
Sort derived list	$O(a \log a)$	$O(a)$ worst case	Standard comparator-based sorting.
Render table	$O(a)$	$O(a)$	Build and measure a fixed number of columns per row.
Rent	$O(n)$	$O(1)$	Lookup plus fixed workflow.
Return	$O(n)$	$O(1)$	Lookup plus transition.
Get revenue	$O(1)$	$O(1)$	Return an immutable value reference.

Table 10.1. Operation costs with respect to fleet size.

ArrayList append is amortized constant time; a particular growth step can copy $O(n)$ references. Its backing storage is $O(n)$ overall [3]. In this application, duplicate search already makes registration linear. Rendering costs also grow with the lengths of the displayed strings, and formatting very large decimals can consume substantial time and memory.

The master fleet is not sorted to satisfy a display preference. Filtering costs $O(n)$, after which sorting only the available subset costs $O(a \log a)$. The complete available-list operation is consequently $O(n + a \log a)$ under the stated assumptions. These costs are appropriate for a small educational fleet.

10.2 Why no Map is implemented now

A hash-based `Map<String, Vehicle>` could provide expected constant-time ID lookup. Keys would have to use the same normalization policy, and duplicate registration would need to remain explicit. If the program kept both a list and a map, every addition and removal would have to keep them consistent.

A map alone would not solve sorted display or filtering: the application would still derive and sort results. The current list avoids maintaining redundant structures and keeps the relationship between data and operations easy to explain. This is a conscious trade-off, not a claim that a list is always the best structure.

10.3 What changes at production scale

Durable storage changes the transaction problem. Multiple users can attempt to rent the same vehicle at the same time; process failure can interrupt a workflow; historical bills may need to survive deletion or changes to fleet records. Such requirements call for persistence, durable transaction boundaries, ownership rules, and concurrency control. Substituting a map for a list does not provide those guarantees.

The present separation still helps: a service boundary already exists, and the UI does not own monetary logic. However, production evolution would be an explicit design extension rather than an assumption that the current in-memory sequence is sufficient.

Presentation takeaway. Use the cost table to explain why linear lookup is reasonable now, then distinguish faster lookup from the much larger problem of durable, concurrent rental management.

Demonstration and Presentation Walkthrough

11.1 A concise explanation of the project

The project can be introduced as follows: it manages a mixed vehicle fleet through a Java console, with an abstract `Vehicle` owning common state, concrete types calculating different subtotals, and `RentalCompany` applying discounts and coordinating rentals. Bills are immutable, money is decimal, and failed operations preserve revenue and availability. The same design is provided in modular and standalone-source forms.

This opening identifies the business problem and the relevant OOP mechanisms without beginning with the list of menu choices. After this introduction, show Figure 2.1, then trace one rental using Figure 4.2. Follow with a concrete price and a rejected duplicate rental.

11.2 Suggested ten-minute presentation structure

Time	Topic	Evidence to show
0–1 min	Problem and scope	One operator, three categories, in-memory model.
1–3 min	Architecture and hierarchy	Figures 2.1 and 2.2.
3–5 min	Pricing and transaction	Car subtotal; 7/8-day boundary; Figure 4.3.
5–8 min	Live demonstration	Successful rental, duplicate rejection, return, discount, revenue.
8–9 min	Design decisions and tests	Immutability, list ownership, 113 checks, standalone release.
9–10 min	Limitations and conclusion	No persistence or concurrency; clear extension boundaries.

Table 11.1. A suggested presentation order; the team may divide speaking roles as preferred.

11.3 Live demonstration sequence

Start a fresh process so all six sample vehicles are available and revenue is zero. The following sequence exposes the key behavior with arithmetic that is easy to verify aloud.

1. Choose 1. Explain that the table is sorted by base daily rate and shows INR amounts. Point out that a car's fee-inclusive rate exceeds its base rate.
2. Choose 2, enter CAR-001, and enter 3 days. Explain $(50 + 15) \times 3 = 195$ and why no discount applies.
3. Choose 1 again. CAR-001 is absent because it is rented.

4. Attempt the same rental again. Explain that the operation is rejected before accounting changes.
5. Choose 4. Revenue remains INR 195.00.
6. Choose 3 and return CAR-001. Choose 1 to show that it reappears. Revenue is unchanged by the return.
7. Rent BIKE-001 for 8 days. Explain subtotal 160.00, discount 16.00, and total 144.00.
8. Rent TRUCK-001 for 3 days. Explain subtotal and total 450.00, including the daily surcharge.
9. View revenue: INR 789.00. Relate that number to the three accepted bill totals.
10. Enter letters for a numeric prompt, try an invalid duration, and search a missing ID. Show recovery, then exit with 0.

11.4 The five-day bike example

For the user's five-day example, BIKE-001 has base rate INR 20.00 and no surcharge. It therefore produces the following receipt content. It is a worked application example, not an additional separately counted test result.

RENTAL BILL

```
-----
Vehicle ID:      BIKE-001
Model:           Swift City 125
Type:            Bike
Rental days:     5

Subtotal (with fees):      INR 100.00
Discount:                  INR 0.00
-----
Total:                      INR 100.00
```

Rental successful. Vehicle is now rented.

Listing 11.1. Receipt content for BIKE-001 rented for five days.

The receipt's five days are a user-specified duration, not dates stored in the system. Discount zero is the correct application of the strict greater-than-seven policy. The success message follows completion of the service call; if the rental fails, a successful bill is not returned to the console.

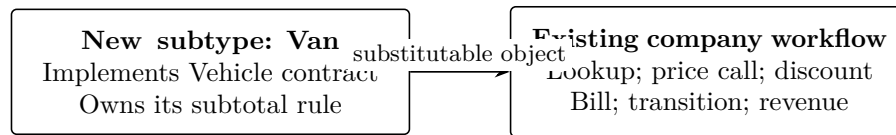
11.5 Explaining a rejected duplicate

After renting a vehicle, explain the state as “available is false.” A second request is a validly formatted input but an invalid business operation. The service reports that the vehicle is already rented. No second subtotal is accepted into revenue, and no second successful result is returned. Returning the vehicle changes availability back to true, permitting a later independent rental.

This example is useful because it connects encapsulation, state transitions, exception handling, and accounting in one visible interaction. It is stronger evidence than merely showing the correct number on a first receipt.

11.6 Explaining extension without implementing it

A hypothetical Van would extend Vehicle, supply a display name, and override the pricing method `calculateRentalCost`. It would preserve the same duration and money contracts. The company would still look it up, call the base method, apply the same discount, and update revenue using the returned bill. The UI would need a construction choice to let the operator add a van.



Construction choices change to expose the category.
 The common rental algorithm does
 not gain a Van-specific pricing branch.

Figure 11.1. The intended extension boundary, shown as a proposed future category rather than delivered functionality.

Presentation takeaway. In a demonstration, show both correct results and correct refusals. Then connect what the audience saw to the responsible classes and the governing invariant.

Limitations, Future Work, and Conclusion

12.1 Deliberate limitations

The application is in-memory, single-process, single-user, console-based, and non-persistent. Ending the process discards registration changes, availability changes, and revenue. No customer identity, rental timestamps, due dates, reservations, payment gateway, persistent bill history, authentication, database, or remote API is implemented.

The company holds references to mutable vehicle objects, so the design assumes cooperating code and one owning company in the application flow. It does not prevent a caller in the unnamed package from invoking package-private transitions or reusing an object across company instances. The service is not thread-safe. These are precisely stated boundaries rather than hidden claims of stronger protection.

The interface supports ordinary terminal output but does not implement terminal-width negotiation, advanced Unicode cell-width measurement, or paging. Very long labels can widen a table, and very large numbers remain subject to practical memory and display limits. The formatting policy is intended for the project's small educational fleet.

12.2 Future work with concrete design consequences

Extension	Required design work
Customer records	Associate a rental with a validated customer identity; define ownership and privacy requirements.
Rental history	Store accepted bill records with stable identifiers and a query API.
Persistent database	Introduce durable storage and transactional guarantees for state and accounting.
Reservation dates	Replace the simple boolean-only availability question with overlap and scheduling rules.
Payment processing	Separate accepted charges from payment authorization and settlement; redefine revenue timing explicitly.
GUI or REST interface	Add another presentation adapter while retaining domain and service rules.
More vehicle types	Add a contract-preserving subtype and the necessary construction choices.
Configurable pricing	Extract policies only when actual variability requires them; retain deterministic rounding.
Concurrent operators	Coordinate lookup, availability, transition, and accounting atomically.

Table 12.1. Possible extensions and the new responsibilities they would introduce.

12.3 Conclusion

The project demonstrates that a small Java application can be both straightforward to run and deliberately structured. Common state lives in `Vehicle`; category prices live in its concrete subclasses; company policy and accounting live in `RentalCompany`; completed results live in `RentalBill`; and terminal interaction lives in `ConsoleMenu`. `Main` and `VehicleRentalApp` assemble these parts without taking over their responsibilities.

The most important correctness choices are validated construction, normalized immutable identity, guarded availability transitions, exact decimal money, and preparation before transaction mutation. The most important maintainability choices are a compact polymorphic contract, one owner for each shared policy, isolated presentation, and mechanically consistent distribution sources.

The verification evidence supports the documented cases, while the stated limits prevent educational guarantees from being confused with production transaction guarantees. The result is a complete Object Oriented Programming mini project whose concepts can be explained from its behavior, traced through its classes, and demonstrated through a short repeatable terminal session.

Presentation takeaway. The central message is simple: good object-oriented structure makes responsibility visible. The project remains portable because its logical classes do not depend on frameworks or on being stored in separate files.

Viva and Presentation Questions

1. Why is `Vehicle` abstract?

It defines common rentable state and operations while leaving category-specific subtotal behavior to concrete classes. Instantiating a generic `Vehicle` would leave that behavior underspecified. An abstract method makes the required extension point explicit.

2. Where exactly is polymorphism used?

`RentalCompany` stores and retrieves `Vehicle` references and calls `calculateRentalCost(days)`. The actual `Car`, `Bike`, `Truck`, or other valid subtype supplies the override selected at runtime. The console also uses the same contract to obtain fee-inclusive daily display prices.

3. Why does the add-vehicle menu contain a switch?

The operator selects which type of object to construct. That construction decision is different from implementing pricing with type codes. Once constructed, the object supplies its price through polymorphism; the company does not branch by category.

4. Why are there getters but no setters?

Presentation and service collaborators need to read selected values. They do not need arbitrary mutation of identity, model, rate, or availability. State changes are expressed by meaningful guarded transitions, and immutable values remain safe to read.

5. Does `final` make `Vehicle` immutable?

No. Some of its fields and methods are `final`, but its availability boolean changes through transitions. `Final` references prevent reassignment; immutability is a property of the whole reachable state and API. `RentalBill`, whose complete stored values are immutable and `final`, is an immutable result object.

6. Why is a bill created before calling `rent()`?

Its constructor may reject inconsistent amounts or a malformed subtotal. Preparing it before mutation prevents those ordinary failures from leaving a vehicle unavailable or revenue changed. It is only returned after the successful commit section.

7. Why validate duration in more than one layer?

The company validates before a transaction, while the shared pricing helper protects direct pricing calls. Both invoke the same authoritative rule, `validateDays`; the rule is not independently redefined in each layer. A caller bypassing the UI still receives validation.

8. Why use `BigDecimal` instead of `double`?

The inputs and output policy are decimal. `BigDecimal` represents exact decimal inputs and supports explicit rounding to two places. The project avoids converting money to binary floating point and defines when normalization and discount rounding occur.

9. Is the discount applied at seven days?

No. The condition is strictly greater than seven. A seven-day bike rental at INR 20/day costs INR 140.00; an eight-day rental has subtotal INR 160.00, discount INR 16.00, and total INR 144.00.

10. Why does returning not change revenue?

The program recognizes an accepted rental charge when renting succeeds. A return only restores availability. It is not a new charge, a refund, or a recalculation of a date-based final bill.

11. Does returning a new list make all vehicles immutable?

No. The container is independent, but its elements are references to the same domain objects. Sorting or clearing it cannot alter master membership, while the referenced vehicles retain their current domain state. Public queries and package-private transitions define the intended access boundary.

12. Why `ArrayList` instead of a map?

The required fleet is small, and linear lookup is easy to inspect. A map could improve lookup for a larger fleet, but maintaining another structure has consistency costs. The current choice is adequate under the stated scale; it is not a universal performance claim.

13. Is the rental method atomic?

It is organized to preserve state on expected failures in the single-threaded flow. It is not a database transaction, a lock, or crash-safe persistence. Concurrent rentals would require additional coordination around the whole use case.

14. Why are concrete categories final?

They are not intended to serve as intermediate extension frameworks. Final communicates that their current meaning is stable. A new category extends the deliberately designed Vehicle contract instead.

15. How can one Java file contain the whole application?

The file contains one public entry-point class named VehicleRentalApp plus package-private top-level collaborators. Multiple logical classes can share one compilation unit. The class architecture survives even though physical files have been combined.

16. What proves that the release is independent?

The README records copying only its source to an empty directory, compiling and running it, deleting generated class files, and repeating the process. Tests and console-parity checks provide additional evidence that the distribution contains the same behavior.

17. Are INR values converted from another currency?

No. The sample amounts are denominated directly in INR. The CLI change updated denomination and presentation; no exchange-rate computation was introduced. The rates are illustrative mini-project data.

18. What is the strongest design decision?

A useful answer is the division between subtype subtotal calculation and company transaction policy. It allows pricing variation without duplicating discounts, and it creates a clear sequence in which a validated bill becomes the source of the accepted revenue amount.

Glossary and Source Navigation

Table B.1. Key terms in the context of this project.

Term	Meaning here
Abstraction	The common Vehicle contract that hides which category implements pricing.
Encapsulation	Protecting identity, money, collection membership, and legal transitions through deliberate APIs.
Invariant	A property that valid application state and operations must preserve, such as positive rental duration.
Precondition	A requirement before an operation succeeds, such as availability before renting.
Postcondition	The result of successful execution, such as a rented state and increased revenue.
Dynamic dispatch	Selection of a concrete overriding method based on an object's runtime type.
Immutable value	Data whose value cannot be altered after creation; strings, decimals, and the bill's stored result.
Snapshot	Copied descriptive and monetary values that do not follow later vehicle state changes.
Shallow copy	A new container retaining references to existing objects rather than cloning those objects.
Subtotal	Category-specific charge including daily fees, before the company discount.
Discount	The rounded reduction applied to subtotal when duration exceeds seven days.
Revenue	Sum of accepted final rental totals in the current process.
HALF_UP	The explicit rounding mode used when normalizing rates and rounding discounts.
UNNECESSARY	A scale check that rejects amounts requiring a changed numerical value to fit two decimals.
Optional	An explicit representation of a search result that may be absent.
Amortized cost	Average cost over a sequence of operations, accounting for occasional larger array growth.

Term	Meaning here
Standalone source	One Java source file that contains all application classes required for compilation.

B.1 Where to find an implementation detail

Question	Source location
How is identity validated and normalized?	Vehicle.java: constructor, requireText, normalizeId.
How are days and transitions protected?	Vehicle.java: validateDays, rent, returnToFleet.
Where are fees defined?	Car.java and Truck.java: named daily constants.
Where is the bike formula?	Bike.java: calculateRentalCost.
Where is the common discount?	RentalCompany.java: named constants and rentVehicle.
Where is the final total derived?	RentalBill.java: constructor.
Where is revenue changed?	RentalCompany.java: the commit section of rentVehicle.
Where is INR formatting?	ConsoleMenu.java: formatAmount, formatMoney, printCharge, table output.
Where are menu errors handled?	ConsoleMenu.java: run, readInteger, readDailyRate.
Where is the fleet created?	Main.java: populateSampleFleet; equivalent release entry point.
Where are verification scenarios?	RentalCompanyTest.java: six focused scenario methods.

Requirement-to-Design Traceability

Table C.1. Traceability from requirement to implementation and verification evidence.

Requirement	Implementation owner	Verification evidence
Valid vehicle construction	Vehicle constructor	Null, blank, negative, and normalized-value checks.
Unique IDs	RentalCompany.addVehicle	Case-insensitive duplicate registration rejection.
Search all fleet states	RentalCompany.findVehicleById	Existing, absent, padded, and mixed-case searches.
Availability listing	RentalCompany plus ConsoleMenu	Rented exclusion, returned reappearance, empty fleet.
Sorted presentation	ConsoleMenu comparator and derived list	Display order and unchanged master ordering.
Subtype pricing	Car, Bike, Truck overrides	Three-day totals and custom subtype check.
Long-rental discount	RentalCompany.rentVehicle	Exactly seven days and eight days.
Correct money scale	Vehicle, service, and bill validation	Fractional rate normalization and malformed subtotal rejection.
Controlled transitions	Vehicle.rent and returnToFleet	Duplicate rental and duplicate return rejection.
Successful immutable bill	RentalBill and company commit order	Snapshot values after return and later rental.
Correct revenue	RentalCompany.totalRevenue assignment	Successful totals; unchanged values after failures and returns.
Robust console	ConsoleMenu parsing and exception handling	Malformed numbers, invalid rates, unknown IDs, and EOF.
Single-file portability	VehicleRentalApp release assembly	Two clean standalone builds plus matching scenario output.
Dependency-free tests	RentalCompanyTest	Ordinary Java execution reports 113 passed checks.

Bibliography

- [1] Project authors, *Vehicle Rental Management System: README and Java Sources*, supplied project documentation and implementation, INR console revision, September 2026. Primary evidence for application behavior, design, and recorded verification.
- [2] Oracle, *Java SE 25 API: BigDecimal*. <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/math/BigDecimal.html>. Decimal representation, scale, equality, and rounding operations.
- [3] Oracle, *Java SE 25 API: ArrayList*. <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/ArrayList.html>. Collection behavior and amortized append costs.
- [4] Oracle, *The Java Language Specification, Section 7.6: Top Level Class and Interface Declarations*. <https://docs.oracle.com/javase/specs/jls/se25/html/jls-7.html#jls-7.6>.
- [5] Oracle, *The Java Language Specification, Section 15.12.4.4: Locate Method to Invoke*. <https://docs.oracle.com/javase/specs/jls/se25/html/jls-15.html#jls-15.12.4.4>.
- [6] Google, *Google Java Style Guide*. <https://google.github.io/styleguide/javaguide.html>. A naming and organization reference; the project intentionally uses four-space indentation and a multiple-class release source.
- [7] Martin Fowler, *Tell Don't Ask*, 2013. <https://martinfowler.com/bliki/TellDontAsk.html>. Behavior and data ownership as a design heuristic.